SciPy.stats

```
from scipy import stats
```

Define a probability distribution:

```
rv = stats.norm(loc=5, scale=2.5)
rv = stats.beta(a=1, b=2)
```

Obtain Statistical Data:

```
rv.mean()
rv.var()
rv.median()
rv.expspect()
rv.stats()      # Mean and Variance
```

Common methods for continuous RVs:

`rv.rvs(size=n)`	Random Variates
`rv.pdf(x)`	Probability Density Function
`rv.cdf(x)`	Cumulative Distribution Function
`rv.ppf(q)`	Inverse CDF

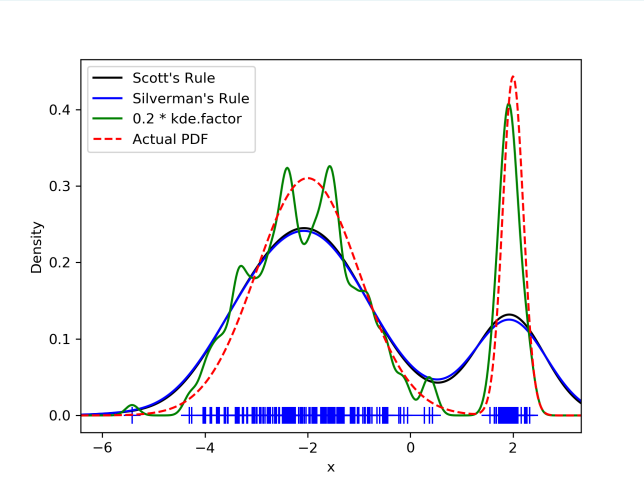
Maximum-Likelihood Estimation:

```
(loc, scale) = stats.norm.fit(data)
```

Kernel density estimation:

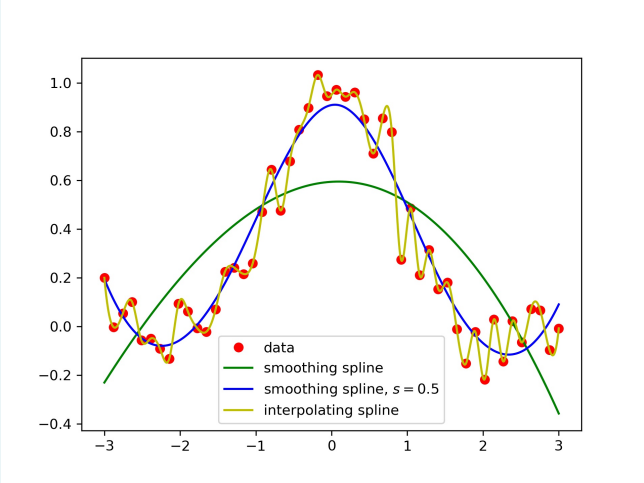
$$\hat{f}(t) = \frac{1}{nh}\sum_{i=1}^{n} k\left(\frac{t-x_i}{h}\right), \quad k(t) = \frac{1}{\sqrt{2\pi}}\exp\left(-\frac{1}{2}t^2\right)$$

```
kde1 = stats.gaussian_kde(x)
kde2 = stats.gaussian_kde(x, bw_method="silverman")
kde3 = stats.gaussian_kde(x, bw_method=0.2*kde.factor)

plt.plot(x, kde1(x), "k-", label=r'Scotts Rule' )
```

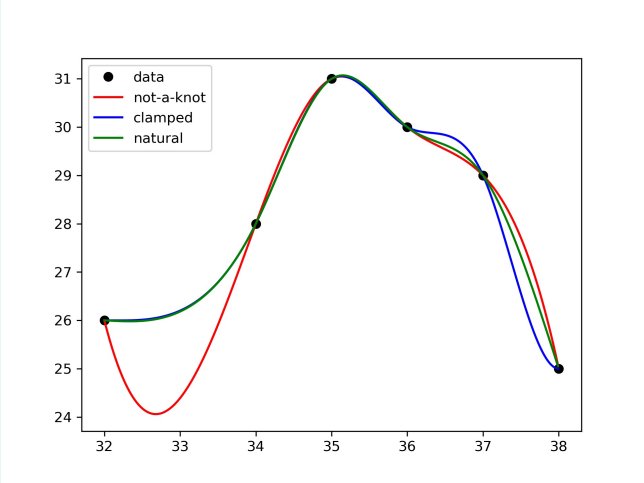


SciPy.interpolate

```
from scipy.interpolate import UnivariateSpline

x = np.linspace(-3, 3, 50)
y = np.exp(-x**2) + 0.1 * np.random.randn(50)
plt.plot(x, y, 'ro', label='data')

# Create UnivariateSpline object, smoothing=0.5
spl = UnivariateSpline(x, y, k=3, s=0.5)
xs = np.linspace(-3, 3, 1000)
plt.plot(xs, spl(xs), 'b-', label='smoothing spline')
```



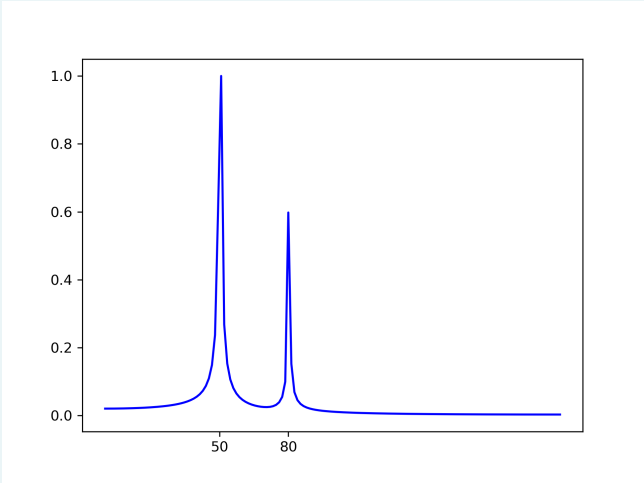

```
from scipy.interpolate import CubicSpline
cs = CubicSpline(x, y)
plt.plot(xs, cs(xs), 'r-', label='not-a-knot')

cs = CubicSpline(x, y, bc_type='natural')
plt.plot(xs, cs(xs), 'g-', label='natural')
```



SciPy.fft

$$y_k = \sum_{n=0}^{N-1} e^{-2\pi i \frac{kn}{N}} x_n$$

```
from scipy.fft import *
from scipy.signal import blackman

N = 300      # number of sample points
T = 1/400    # Sample Rate
x = np.linspace(0, N*T, N)
y = np.sin(50*2*np.pi*x) + 0.5*np.sin(80*2*np.pi*x)
y *= blackman(N)        # Apply filter

xf = fftfreq(N, T)
yf = np.abs(fft(y))

plt.plot(xf, yf, ".b-")
```



SciPy.integrate

```
from scipy.integrate import quad, solve_ivp
```

$$\int_a^b f(x)\,\mathrm{d}x, \qquad \begin{cases} \mathbf{y}'(t) = f(t,\mathbf{y}), & t \in I \\ \mathbf{y}(0) = y_0 \end{cases}$$

```
# 1d-integration:
f = lambda x: x**2
integrate.quad(f, a, b)
```

```
# Solve initial value problem
f = lambda t, y: -y
ivl = (0, 10)
y0 = 1

sol = solve_ivp(f, ivl, y0, method="RK45", t_eval=None)
sol.y   # Solution
```